

EF Core 8.0 Guided Hands-On Exercises — Answer Key

Lab 3: Using EF Core CLI to Create and Apply Migrations

Scenario	The retail store's database needs to be created based on the models you've defined. You'll use EF Core CLI to generate and apply migrations.
Objective	Learn how to use EF Core CLI to manage database schema changes.

Step 1: Install EF Core CLI

```
dotnet tool install --global dotnet-ef
```

Installs the dotnet-ef global tool, which provides the command-line interface for generating and applying migrations. Verify with:

```
dotnet ef --version
```

Note: your project also needs the Microsoft.EntityFrameworkCore.Design package (installed in Lab 1) for these commands to work.

Step 2: Create Initial Migration

```
dotnet ef migrations add InitialCreate
```

What happens: EF Core inspects ApplicationDbContext and its DbSet properties/entity classes, then generates a new **Migrations** folder containing three files: a migration class with Up() and Down() methods (Up creates the schema, Down reverts it), a designer file, and a snapshot of the current model. The Up() method effectively represents the CREATE TABLE statements for Products and Categories, generated automatically from your C# classes — this is the ORM mapping made concrete.

Step 3: Apply Migration to Create Database

```
dotnet ef database update
```

Executes the pending migration(s) against the target SQL Server instance. If the database doesn't exist yet, EF Core creates it first, then runs the Up() method to create the Products and Categories tables (with the CategoryId foreign key linking them).

Step 4: Verify in SQL Server

Open SQL Server Management Studio (SSMS) or Azure Data Studio, connect to the server/instance used in the connection string, and expand:

```
Databases > RetailInventoryDb > Tables
    dbo.Products
    dbo.Categories
    dbo.__EFMigrationsHistory
```

Note: EF Core also creates a `__EFMigrationsHistory` table automatically. It tracks which migrations have already been applied, so running `dotnet ef database update` again is safe — EF Core will only apply migrations that haven't run yet.

Key Takeaway

The EF Core CLI turns your `C#` model into real database schema and keeps a versioned history of every schema change, which is essential for team collaboration and deployments.